

CS224N – Bài giảng 3: Neural Network Foundations

Lecture note: Word Vectors, Matrix Calculus & Backpropagation

Ghi chú học tập – NLP with Deep Learning

Stanford University

1 Non-linearities

Vì sao cần phi tuyến? Mạng làm *function approximation*. Không có phi tuyến thì $\mathbf{W}_2\mathbf{W}_1\mathbf{x} = \mathbf{W}\mathbf{x}$ — nhiều tầng tuyến tính co lại thành *một* phép biến đổi, **không thêm sức mạnh biểu diễn**. Có phi tuyến $\Rightarrow$ xấp xỉ được hàm phức tạp bất kỳ.

Lịch sử: neuron ngưỡng (1940s) phẳng $\Rightarrow$ gradient = 0, học khó. Cần hàm *có độ dốc* để làm gradient-based learning.

Các hàm chính:

Hàm	Công thức	Đạo hàm
Logistic (sigmoid)	$\sigma(z) = \frac{1}{1+e^{-z}}$	$\sigma(z)(1 - \sigma(z))$
Tanh	$\tanh(z) = 2\sigma(2z) - 1$	$1 - \tanh^2(z)$
ReLU	$\max(z, 0)$	$\mathbf{1}[z > 0]$
Leaky/Parametric ReLU	$\max(az, z)$	$\mathbf{1}[z > 0] + a \mathbf{1}[z < 0]$
GELU	$z \Phi(z) \approx z \sigma(1.702z)$	$\Phi(z) + z \phi(z)$
SiLU / Swish	$z \sigma(z) / z \sigma(\beta z)$	$\sigma(z)[1 + z(1 - \sigma(z))]$

Ghi nhớ:

- *Sigmoid*: output không âm (không zero-centered), dễ bão hoà $\Rightarrow$ vanishing gradient.
- *Tanh*: chỉ là sigmoid co giãn về $[-1, 1]$.
- *ReLU*: mặc định nên thử đầu tiên — train nhanh, gradient backflow tốt (dốc = 1); nhược điểm “dead zone” âm.
- *GELU/SiLU/Swish*: giống ReLU nhưng không chết; phổ biến trong Transformers.
- *GLU/SwiGLU*: $\text{SwiGLU}(x) = (xV + c) \otimes \text{Swish}_\beta(xW + b)$; dùng trong Llama3, Qwen3.

2 Matrix Calculus

Gradient ($f : \mathbb{R}^n \rightarrow \mathbb{R}$): vector $\nabla_{\mathbf{x}}f$ cùng cỡ số đầu vào.

Jacobian ($\mathbf{f} : \mathbb{R}^n \rightarrow \mathbb{R}^m$): ma trận $\in \mathbb{R}^{m \times n}$, $(\frac{\partial \mathbf{f}}{\partial \mathbf{x}})_{ij} = \frac{\partial f_i}{\partial x_j}$.

Chain rule nhiều biến = *nhân các Jacobian*. Kiểm tra khớp chiều để gỡ lỗi.

Kho Jacobian cần nhớ:

Elementwise $\mathbf{h} = f(\mathbf{z})$	$\partial \mathbf{h} / \partial \mathbf{z} = \text{diag}(f'(\mathbf{z}))$
$\mathbf{z} = \mathbf{W}\mathbf{x} + \mathbf{b}$ theo $\mathbf{x}$	$\partial \mathbf{z} / \partial \mathbf{x} = \mathbf{W}$
$\mathbf{z} = \mathbf{W}\mathbf{x} + \mathbf{b}$ theo $\mathbf{b}$	$\partial \mathbf{z} / \partial \mathbf{b} = \mathbf{I}$
$s = \mathbf{u}^\top \mathbf{h}$ theo $\mathbf{u}$	$\partial s / \partial \mathbf{u} = \mathbf{h}^\top$
$s = \mathbf{u}^\top \mathbf{h}$ theo $\mathbf{h}$	$\partial s / \partial \mathbf{h} = \mathbf{u}^\top$

Kết quả cho mạng NER (với $\delta = \mathbf{u}^\top \odot f'(\mathbf{z})^\top = \text{error signal dùng chung}$):

$$\frac{\partial s}{\partial \mathbf{b}} = \delta, \quad \frac{\partial s}{\partial \mathbf{W}} = \delta^\top \mathbf{x}^\top \text{ (outer product),} \quad \frac{\partial s}{\partial \mathbf{x}} = \mathbf{W}^\top \delta^\top.$$

Ý tưởng chính: tính δ *một lần*, tái sử dụng cho mọi tham số ($\odot = \text{tích Hadamard, elementwise}$).

Jacobian form vs. Shape convention:

- *Jacobian form*: đúng toán học, làm chain rule dễ (nhân ma trận). $\partial s / \partial \mathbf{W}$ là $1 \times nm$.
- *Shape convention*: gradient cùng shape với tham số $\Rightarrow$ trừ SGD dễ. $\partial s / \partial \mathbf{W}$ là $n \times m$.
- **Quy tắc**: bài tập CS224N theo shape convention. Dùng *Jacobian để tính*, dùng *shape convention để trình bày*. Error signal δ tới tầng ẩn có cùng số chiều với tầng ẩn đó.

3 Backpropagation

SGD: $\theta^{\text{new}} = \theta^{\text{old}} - \alpha \nabla_\theta J(\theta)$, $\alpha = \text{learning rate}$. Trong deep learning θ gồm cả word vectors.

Backprop = 2 ý tưởng: (1) chain rule; (2) lưu kết quả trung gian để không tính lại.

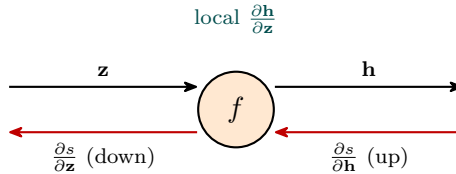
Computation graph: source nodes = input, interior nodes = phép toán.

- *Forward pass*: tính giá trị hàm số, lưu giá trị trung gian.
- *Backward pass*: chạy ngược cùng đồ thị, khởi tạo $\partial s / \partial s = 1$, truyền gradient.

Quy tắc tại mỗi nút:

$$[\text{downstream gradient}] = [\text{upstream gradient}] \times [\text{local gradient}]$$

Local gradient = gradient của output nút theo input nút. Nhiều đầu vào $\Rightarrow$ nhiều downstream gradient (mỗi cái nhân cùng upstream).



Trực giác từng loại nút:

- $+$: *distributor* — gửi cùng gradient tới mọi đầu vào.
- $\max$: *router* — gửi hết cho đầu vào lớn nhất, 0 cho phần còn lại.
- $\times$: *switcher* — upstream $\times$ hệ số forward của phía đối diện.

Outward branches: biến có nhiều nhánh đi ra $\Rightarrow$ gradient là **tổng** các nhánh:

$$\frac{\partial s}{\partial x} = \sum_{y \in \text{children}(x)} \frac{\partial s}{\partial y} \frac{\partial y}{\partial x}.$$

Hiệu quả: tính phần dùng chung (δ) một lần. Backward pass có **cùng Big-O** với forward pass — nếu chậm hơn nghĩa là đang tính lại thừa. Cơ chế này tự động hoá thành *autodiff* (PyTorch/JAX).

Gradient check (numeric): $\frac{\partial f}{\partial \theta_i} \approx \frac{f(\theta + h\mathbf{e}_i) - f(\theta - h\mathbf{e}_i)}{2h}$, $h \approx 10^{-4}$. Dễ cài đúng nhưng chậm; dùng để kiểm tra layer tự viết.